

Week 3 Report

Building the RAG-Based Code Q&A Application

Objective

Build one progressively-developed application demonstrating the full pipeline: **Application** → **API** → **Retrieval/RAG** → **Chunking** → **Embeddings** → **Vector Similarity** → **Relevant Context** → **LLM** → **Response**, using Ollama + Code Llama + APIs + Services + Orchestration + Docker.

Application Chosen: Code Q&A Assistant

An app that answers natural-language questions about a small sample codebase by retrieving relevant code chunks and passing them as context to Code Llama.

Exercise 1 — Basic LLM Connection

A minimal script (`exercise1_basic.py`) sends a prompt to Ollama's HTTP API (`/api/generate`) and prints Code Llama's response, proving the basic pipeline: `User` → `App` → `API` → `Ollama` → `Code Llama` → `Response` .

Exercise 2 — Knowledge Base (Chunking + Embeddings)

A small sample codebase was created to act as the "knowledge" the app answers questions about:

`auth.py`

```
import hashlib

def hash_password(password: str) -> str:
    """Hashes a password using SHA-256."""
    return hashlib.sha256(password.encode()).hexdigest()

def authenticate_user(username: str, password: str, users_db: dict) -> bool:
    """Checks if the given username/password matches a stored user record."""
    if username not in users_db:
```

```
        return False
    return users_db[username] == hash_password(password)
```

payment.py

```
from auth import authenticate_user

def process_payment(username: str, password: str, amount: float, users_db: dict) -> str:
    """Processes a payment only if the user is authenticated."""
    if not authenticate_user(username, password, users_db):
        return "Authentication failed. Payment denied."
    if amount <= 0:
        return "Invalid payment amount."
    return f"Payment of ${amount:.2f} processed for {username}."
```

registration.py

```
from auth import hash_password

def register_user(username: str, password: str, users_db: dict) -> str:
    """Registers a new user by storing their hashed password."""
    if username in users_db:
        return "Username already exists."
    users_db[username] = hash_password(password)
    return f"User {username} registered successfully."
```

Each file was split into chunks and converted into embeddings (vector representations of meaning) using Ollama's `nomic-embed-text` model, then saved to `knowledge_base.json`.

Full Knowledge Base Contents

File	Chunk #	Text	Embedding
------	---------	------	-----------

auth.py	0	<pre> import hashlib def hash_password(password: str) -> str: """Hashes a password using SHA-256.""" return hashlib.sha256(password.encode()).hexdigest() def authenticate_user(username: str, password: str, users_db: dict) -> bool: """Checks if the given username/password matches a stored use </pre>	768 dimensions
auth.py	1	<pre> r record.""" if username not in users_db: return False return users_db[username] == hash_password(password) </pre>	768 dimensions
payment.py	0	<pre> from auth import authenticate_user def process_payment(username: str, password: str, amount: float, users_db: dict) -> str: """Processes a payment only if the user is authenticated.""" if not authenticate_user(username, password, users_db): return "Authentication failed. Payment den </pre>	768 dimensions
payment.py	1	<pre> ied." if amount <= 0: return "Invalid payment amount." return f"Payment of \${amount:.2f} processed for {username}." </pre>	768 dimensions
registration.py	0	<pre> from auth import hash_password def register_user(username: str, password: str, users_db: dict) -> str: """Registers a new user by storing their hashed password.""" if username in users_db: return "Username already exists." users_db[username] = hash_password(password) return </pre>	768 dimensions

registration.py	1	f"User {username} registered successfully."	768 dimensions
-----------------	---	---	-------------------

Note: embedding vectors are 768-dimensional lists of numbers representing each chunk's meaning — shown here as dimension count only, since the raw numbers are not human-readable.

Exercise 3 — Retrieval + RAG

When a question is asked, it is converted to an embedding and compared (via cosine similarity) against every chunk in the knowledge base; the most similar chunks are retrieved and given to Code Llama as context.

Demonstration: With vs. Without RAG

Question: "Which file handles user authentication and how does it work?"

WITHOUT RAG (no retrieved context):

"User authentication is typically handled by a file called `auth.php` or `authentication.php` ... includes a database connection and a SQL query..."

❌ **Hallucinated** — invented a fictional PHP/SQL file. This codebase has no PHP or database at all.

WITH RAG (retrieved context from `registration.py`, `payment.py`):

"The `auth` module in the `payment.py` file handles user authentication by using the `authenticate_user()` function... defined in the `auth.py` file..."

✅ **Grounded and accurate** — correctly identified the real function and its real behavior.

Exercise 4 — Services + Orchestration

The application was split into independent services communicating over HTTP APIs:

- **Retrieval Service** (port 8001) — exposes `/retrieve`, returns relevant chunks for a question.
- **App / Orchestration Service** (port 8000) — exposes `/ask`, calls the Retrieval Service, then calls Ollama (the LLM Service, already its own service on port 11434), and returns the final answer.

Verified end-to-end via:

```
curl -X POST http://localhost:8000/ask -H "Content-Type: application/json" \
-d '{"question": "Which file handles user authentication?"}'
```

→ Successfully returned a grounded JSON answer, proving the orchestration works.

Exercise 5 — Dockerization

The App Service and Retrieval Service were each packaged into Docker containers (via individual Dockerfiles) and orchestrated together using `docker-compose.yml`. Ollama/Code Llama itself runs natively on the VM host (not containerized) — a deliberate choice, since containerizing it would require re-downloading the 3.8 GB model inside the container for no functional benefit; the containers reach it over the network via `host.docker.internal`.

Verified working: `docker compose up --build` started both containers successfully, and the same `curl` test above returned a correct, grounded answer — confirming the fully containerized services + host-level Ollama architecture works end-to-end.

Conclusion

All 5 exercises of Week 3 were completed and verified working: a RAG-based Code Q&A application, built progressively from a single script into a multi-service, Dockerized architecture, with clear evidence that retrieval-augmented generation meaningfully improves answer grounding compared to the LLM alone.